

Portfolio Project — Full Guide

Flutter Web + Node/Express/MySQL + Admin Panel
Deployed at bhojrajmishra.com.np

Written for Bhoj Raj Mishra · August 2026

A beginner-friendly walkthrough of every part of the codebase, how it's deployed, and everything built this project.

Table of Contents

1. Part 1 — What This Project Is
2. Part 2 — The Frontend (Flutter) Explained
3. Part 3 — The Backend (Node/Express) Explained
4. Part 4 — The Database
5. Part 5 — Deployment: First Time, Step by Step
6. Part 6 — Deployment: Updating the Live Site
7. Part 7 — Everything Built, in Order (Changelog)
8. Part 8 — Credentials & Reference

Part 1 — What This Project Is

This is a portfolio website that looks and behaves like a small operating-system desktop instead of a normal scrolling page: a menu bar with a live clock, a real weather widget, desktop icons, a dock, and draggable/resizable windows for each section (About, Projects, Experience, Contact, Blog). Every piece of content — your bio, skills, projects, experience, blog posts — lives in a database and is edited through a password-protected **admin panel**, so updating the site never requires touching code or redeploying.

The three big pieces

Piece	What it is	Where it runs
frontend/	A Flutter app compiled to a website (HTML/CSS/JavaScript)	Served as static files — no server process needed to run it, just somewhere to host the files
backend/	A Node.js/Express API that talks to the database, handles login, file uploads, etc.	Runs continuously as its own process
MySQL database	Stores every piece of content	Hosted separately by your web host

How a page load actually works

1. Your browser requests `bhojrajmishra.com.np` → gets the compiled Flutter app (static files).
2. The Flutter app boots and immediately calls the API (e.g. `GET /api/hero` , `GET /api/projects`) to fetch real content.
3. The API (Node/Express) queries MySQL and returns JSON.
4. Flutter renders that JSON as widgets — the About window, the Projects grid, etc.

When you edit something in the admin panel, it's the exact same flow in reverse: the admin form calls the API with a login token, the API writes to MySQL, and the next time anyone loads the site (or the app re-fetches), they see the new content. **No code change, no redeploy, ever needed just to update content.**

Why it's built this way (the technology choices)

- **Flutter for the frontend** — one codebase, one language (Dart) for both the public site and the whole admin panel, with real interactive UI (drag/resize windows) that would be painful to build in a simpler templating approach.
- **Node/Express + MySQL for the backend, with no ORM (no Prisma, no Sequelize)** — just a plain `mysql2` connection pool and hand-written SQL. This was a deliberate late change: your hosting account has a small file-count quota, and ORM tools like Prisma generate large amounts of extra files that didn't fit. Plain SQL uses far fewer files and is arguably easier to actually read line by line as a beginner, since there's no framework magic — every query is visible.
- **A single "site" per resource** — the API and the static files live on the same domain (via alwaysdata's path-based routing), so the browser never has to deal with cross-origin requests.

Part 2 — The Frontend (Flutter) Explained

Everything lives under `frontend/lib/`. It's split into two big ideas: `core/` (shared plumbing — API calls, data models, theme, generic widgets) and `features/` (the actual screens, split into `desktop/` for the public site and `admin/` for the admin panel).

2.1 — `lib/main.dart`

The entry point. It wraps the whole app in a `ProviderScope` (this is what makes Riverpod — the state-management library used throughout — work) and sets up `MaterialApp.router`, which hands off all navigation to `go_router` (see 2.4).

2.2 — `lib/core/models/` — the data shapes

One small file per content type. Each model is a plain Dart class with a `fromJson` (turns the API's JSON response into a Dart object) and usually a `toJson` (turns a Dart object back into JSON to send to the API). These are simple, repetitive, and intentionally boring — that's the point, they just describe the shape of your data:

File	Represents
<code>hero_content.dart</code>	Name, title, subtitle, summary shown in the Hero/menu bar
<code>about_content.dart</code>	Your bio text
<code>skill.dart</code>	One skill (name + category, e.g. "Flutter" / "Mobile Development")
<code>project.dart</code>	One project (title, description, tags, links, featured flag)
<code>experience.dart / education.dart</code>	One work or education entry
<code>social_link.dart</code>	One social platform link + optional admin-set badge text
<code>resume_file.dart / apk_file.dart</code>	The currently-uploaded resume PDF / Android APK
<code>blog_post.dart</code>	One blog post (title, slug, excerpt, Markdown content, published flag)
<code>site_settings.dart</code>	The weather widget's configured city/coordinates
<code>weather.dart</code>	Not from your database — parsed from the live Open-Meteo weather API response
<code>contact_message.dart</code>	One message submitted through the public contact form

2.3 — `lib/core/api/` — talking to the backend

`config.dart`

Defines `apiBaseUrl` — where the API lives. In production this is just `/api` (relative, same domain). For local development you override it: `flutter run --dart-define=API_BASE_URL=http://localhost:3000/api`.

auth_storage.dart + auth_controller.dart

`AuthStorage` saves the admin's login token to the browser's local storage so you stay logged in across page reloads. `AuthController` is a Riverpod `Notifier` holding the current login state (logged in / out / still loading) and exposing `login()` / `logout()`. It also provides a shared `Dio` (the HTTP client library) instance that automatically attaches `Authorization: Bearer <token>` to every request, and automatically logs you out if the server ever responds with 401 Unauthorized.

portfolio_repository.dart

This is the single place that knows the shape of every API call — one method per endpoint (`getHero()`, `updateHero()`, `getProjects()`, `createProject()`, `deleteProject()`, ...). Every screen in the app goes through this class rather than calling `Dio` directly — if an API path ever changes, you only fix it in one place.

weather_repository.dart

A separate, tiny repository that calls Open-Meteo (a free weather API, no key needed) directly from the browser — not through your backend, since it doesn't need authentication or your database.

data_providers.dart

Turns each repository method into a Riverpod `FutureProvider` — e.g. `projectsProvider` fetches and caches the project list. Any widget that does `ref.watch(projectsProvider)` automatically rebuilds when that data changes. After any admin edit, the code calls `ref.invalidate(xProvider)` to force a refetch, which is how "edit in admin → appears on site instantly" actually works under the hood.

2.4 — lib/core/router/app_router.dart

Defines every URL the app understands, using `go_router`. Two big groups:

- `/` → the public desktop (`DesktopHomePage`)
- `/admin/*` → wrapped in a `ShellRoute` that adds the sidebar (`AdminShell`), with one route per admin page (`/admin/hero`, `/admin/projects`, `/admin/blog`, etc.)

It also has a `redirect` function that runs on every navigation: if you try to visit an `/admin/*` page while logged out, it bounces you to `/admin/login`; if you're logged in and land on the login page, it sends you straight to the dashboard.

2.5 — lib/core/theme/app_theme.dart

All the shared colors (`AppColors`) and the app-wide dark `ThemeData`. Also defines `Breakpoints` — the pixel widths used to decide desktop vs. mobile layout.

2.6 — The public site: lib/features/desktop/

This is the most novel part of the codebase — a small window manager, built from scratch.

window_manager.dart

Defines what "windows" exist (`about` , `projects` , `experience` , `contact` , `blog` — each with a title, icon, and default size) and a Riverpod `Notifier` (`WindowManagerNotifier`) holding the list of currently-open windows, each with a position, size, and stacking order (z-index). It exposes methods like `openWindow` , `closeWindow` , `focusWindow` , `moveWindow` , `resizeWindow` , `toggleMaximize` — pure state changes, no UI code.

widgets/window_frame.dart

The actual draggable/resizable chrome around a window: the title bar with traffic-light buttons (red=close, yellow=minimize, green=maximize), a `GestureDetector` that turns mouse-drag into calls to `moveWindow` , and a resize handle in the corner. It also has the `buildWindowContent(id)` function that picks which content widget to show based on the window's id.

widgets/desktop_background.dart, menu_bar.dart, weather_widget.dart, desktop_icon.dart, desktop_icons_column.dart, dock.dart, calendar_tile.dart

The visual "desktop" chrome: a hand-painted gradient wallpaper (not a copied image), the top menu bar with a live clock (`Timer.periodic` updating every 30 seconds), the floating weather card, the right-side icon column (Projects + your social links, each with real brand icons via the `font_awesome_flutter` package), the bottom dock (Resume / About / Projects / Experience / Contact / Blog / Download App), and a small live-updating calendar-style tile used for the Experience icon.

window_content/*.dart

The actual content shown inside each window — `about_window_content.dart` , `projects_window_content.dart` , `experience_window_content.dart` , `contact_window_content.dart` , `blog_window_content.dart` . These are ordinary widgets that read from the providers in 2.3 and lay the data out — nothing window-manager-specific happens here, which is why the exact same widgets are reused for the mobile fallback (below).

desktop_home_page.dart

The top-level page for `/` . Uses a `LayoutBuilder` to check the available width: $\geq 1000\text{px}$ shows the full draggable-window desktop; narrower shows `_MobileHome` — a simple scrollable icon grid where tapping an icon pushes a fullscreen page (`mobile_content_page.dart`) that renders the exact same window-content widget.

2.7 — The admin panel: lib/features/admin/

auth/login_page.dart

The login form. Calls `AuthController.login()` ; on success, `go_router` 's redirect logic (2.4) automatically sends you to the dashboard.

dashboard/admin_shell.dart

The sidebar layout wrapping every admin page — the list of nav links (Dashboard, Hero, About, Skills, Projects, Experience, Education, Social Links, Resume, Messages, Settings, Blog, Android App), plus "View Site" and "Log Out".

dashboard/pages/*.dart

One page per content type, and they all follow the same shape: fetch the current data with a provider, show it in a form or list, and on Save call the matching repository method then invalidate the provider so the change reflects immediately. A few worth calling out specifically:

- **hero_admin_page.dart / about_admin_page.dart / settings_admin_page.dart** — "singleton" forms (there's only ever one Hero, one About, one Settings row).
- **skills_admin_page.dart / experience_admin_page.dart / education_admin_page.dart / projects_admin_page.dart / social_links_admin_page.dart** — list + add/edit dialog pattern, each with delete confirmation (`admin_helpers.dart` 's `confirmDelete`).
- **resume_admin_page.dart / apk_admin_page.dart** — file upload pages, using `file_pick_helpers.dart` to open the browser's file picker and then upload the bytes.
- **blog_admin_page.dart + blog_post_editor_page.dart** — a list page plus a separate full-page editor (title, slug, excerpt, a Markdown textarea with an Edit/Preview toggle, a cover-image upload, and a Published checkbox).
- **contact_admin_page.dart** — the inbox of messages submitted through the public contact form.

Part 3 — The Backend (Node/Express) Explained

Everything lives under `backend/src/`. There is no ORM — every database call is a hand-written, parameterized SQL query through the `mysql2` library. Parameterized means the actual values (like a title you type in) are passed separately from the SQL text via `?` placeholders, which is what prevents SQL-injection attacks.

3.1 — `server.ts` and `app.ts`

`server.ts` is the actual entry point — it starts the HTTP server, listening on whatever port/IP the hosting environment provides (or a sensible default locally). `app.ts` builds the actual Express application: security headers (`helmet`), CORS rules, JSON body parsing, serves the `/uploads` folder as static files, and then registers every route file under its `/api/...` path.

3.2 — `lib/env.ts`

Reads every environment variable the app needs from `.env` (via the `dotenv` package) into one typed `env` object, so the rest of the code never touches `process.env` directly. Throws a clear error at startup if something required is missing.

3.3 — `lib/db.ts`

Creates one shared `mysql2` connection pool (`pool`) from `DATABASE_URL`. Every route imports this same `pool` to run queries. Notably sets `dateStrings: ['DATE']` — without this, MySQL `DATE` columns come back as JavaScript `Date` objects that silently shift by a day depending on the server's timezone; this setting makes them come back as plain `"YYYY-MM-DD"` strings instead, sidestepping the whole problem (this was an actual bug found and fixed during development).

3.4 — `lib/upload.ts`

Configures `multer` (the file-upload library) with three variants — `uploadImage`, `uploadResume`, `uploadApk` — each restricting file type and size, all writing into the same `UPLOADS_DIR`. This constant is computed from `process.cwd()` rather than `__dirname` deliberately — an earlier version used `__dirname`, which pointed at a different (and wrong) folder once the code was compiled for production, silently losing uploaded files on every redeploy. Fixed during this project; see the changelog in Part 7.

3.5 — `lib/dateOnly.ts`

A tiny shared Zod (validation library) schema used by the Experience and Education routes to accept a date string from the frontend and keep only the `YYYY-MM-DD` part — paired with the `db.ts` fix above, dates never pass through a JavaScript `Date` object anywhere in the write path either.

3.6 — middleware/auth.ts and middleware/errorHandler.ts

`requireAuth` is the guard placed on every admin (write) route — it checks for a valid `Authorization: Bearer <token>` header and rejects the request with 401 if missing/invalid. `errorHandler` is Express's catch-all error handler, turning thrown errors (including Zod validation failures) into clean JSON error responses instead of crashing.

3.7 — routes/*.ts — the actual API, resource by resource

Every route file follows the same shape: a public `GET` (no login needed) and one or more admin-only routes (`requireAuth`) for `POST` / `PUT` / `DELETE`. Rather than repeat the same explanation eleven times, here's the pattern once, using `skills.ts` as the example:

```
router.get("/", ...) // anyone can list all skills
router.post("/", requireAuth, ...) // only a logged-in admin can add one
router.put("/:id", requireAuth, ...) // only admin can edit one
router.delete("/:id", requireAuth, ...) // only admin can delete one
```

Every route file:

File	Resource	Notes
<code>auth.ts</code>	Admin login	Rate-limited (10 attempts / 15 min) to slow down password guessing. Checks the password against a bcrypt hash and returns a signed JWT.
<code>hero.ts</code> / <code>about.ts</code> / <code>settings.ts</code>	Hero text / bio / weather location	"Singleton" pattern — always row id=1, using MySQL's <code>INSERT ... ON DUPLICATE KEY UPDATE</code> to create-or-update in one query.
<code>skills.ts</code> / <code>experience.ts</code> / <code>education.ts</code> / <code>social_links.ts</code>	Lists of small records	Standard CRUD, straightforward.
<code>projects.ts</code>	Projects	Tech tags are stored as a JSON-encoded string in one TEXT column and parsed back into a real array before being sent to the frontend.
<code>resume.ts</code> / <code>apk.ts</code>	Resume PDF / Android APK	File upload via multer; only the most recently uploaded row is treated as "current" (returned by <code>ORDER BY uploaded_at DESC LIMIT 1</code>).
<code>blog.ts</code>	Blog posts	Auto-generates a URL slug from the title if you don't set one; the public Blog window filters to <code>isPublished</code> posts client-side, so the admin panel can see drafts.
<code>contact.ts</code>	Contact form	Public submit is rate-limited (5 / 15 min) since public forms are a common spam target; admin can list, mark read, delete.
<code>uploads.ts</code>	Generic image upload	Used by admin forms that need to attach an image (project thumbnail, blog cover) — returns just the uploaded file's URL, not tied to any specific content type.

3.8 — `scripts/seed.ts`

A one-off script (not part of the running server) that creates/updates the single admin login from `ADMIN_EMAIL` / `ADMIN_PASSWORD` in `.env`, and inserts placeholder content into every table if it's empty. Run once on first setup, and again any time you want to change the admin password (it re-hashes and overwrites by email).

Part 4 — The Database

`backend/sql/schema.sql` is the single source of truth for every table — no migration tool, just plain SQL using `CREATE TABLE IF NOT EXISTS`, so it's always safe to re-run the whole file against an existing database. Adding a brand-new column to an existing table needs a separate manual `ALTER TABLE` (this file only handles brand-new tables safely).

Table	Holds
<code>admin_users</code>	The single admin login (email + bcrypt password hash)
<code>hero_content</code>	Name/title/subtitle/summary shown in the menu bar (1 row)
<code>about_content</code>	Bio text (1 row)
<code>skills</code>	Skill list, grouped by category
<code>projects</code>	Project cards
<code>experience / education</code>	Timeline entries
<code>social_links</code>	One row per platform (github, linkedin, email, ...) + optional badge text
<code>resume_files / apk_files</code>	Upload history — latest row = current file
<code>contact_messages</code>	Public contact form submissions
<code>site_settings</code>	Weather widget's city/coordinates (1 row)
<code>blog_posts</code>	Blog posts (title, slug, excerpt, Markdown content, cover image, published flag)

Part 5 — Deployment: First Time, Step by Step

This project is hosted on **alwaysdata** — shared web hosting, not a VPS. That distinction matters: there's no root/sudo access, no editing Nginx config files directly. Instead, "Sites" (static file hosting, or a supervised Node.js process), databases, domains, and SSL certificates are all set up through alwaysdata's web control panel at **admin.alwaysdata.com**. SSH is used only to upload code and run one-off commands.

The split to remember: SSH commands move your *code* onto the server. The admin panel tells alwaysdata what to *do* with that code (which folder is a website, which is a running app, which domain points where, SSL on/off). Neither one can do the other's job.

5.1 — Generate a dedicated SSH key

```
ssh-keygen -t ed25519 -f ~/.ssh/alwaysdata_portfolio -N "" -C "portfolio-deploy"
```

Creates a private key (never share) and a public key (safe to share — this is what gets installed on the server so you don't have to type a password every time).

5.2 — Install the key on the server (one-time, needs the password once)

```
ssh bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net \  
"mkdir -p ~/.ssh && chmod 700 ~/.ssh && \  
echo 'ssh-ed25519 AAAA...your key here... portfolio-deploy' >> ~/.ssh/authorized_keys && \  
chmod 600 ~/.ssh/authorized_keys"
```

From then on, every command uses the key:

```
ssh -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes \  
bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net "whoami"
```

-o IdentitiesOnly=yes tells SSH to only try this specific key — without it, SSH sometimes tries other keys on your machine first and can temporarily lock you out with "too many authentication failures."

5.3 — Create the project folder on the server

```
ssh -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes \  
bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net \  
"mkdir -p /home/bhojrajmishra/portfolio/frontend /home/bhojrajmishra/portfolio/backend/uploads"
```

5.4 — Create the MySQL database (admin panel, not SSH)

admin.alwaysdata.com → **Databases** → **MySQL** → **Add a database**. This gives you a database name, username, and password — write them down, you'll need them for `.env`.

5.5 — Build and upload the backend

```
cd backend
npm run build          # compiles TypeScript into backend/dist/

tar --exclude='node_modules' --exclude='.env' --exclude='uploads' -czf backend.tar.gz .

scp -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes backend.tar.gz \
    bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net:/home/bhojrajmishra/portfolio/backend.tar.gz

ssh -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net \
    "cd /home/bhojrajmishra/portfolio/backend && tar xzf ../backend.tar.gz && rm ../backend.tar.gz"
```

5.6 — Create the production `.env` file

Written locally, then uploaded directly (never through git, since it holds real secrets):

```
DATABASE_URL="mysql://<dbuser>:<dbpassword>@mysql-bhojrajmishra.alwaysdata.net:3306/<dbname>"
JWT_SECRET="<a long random string, e.g. from `openssl rand -hex 32`>"
JWT_EXPIRES_IN="12h"
ADMIN_EMAIL="you@example.com"
ADMIN_PASSWORD="<a strong password>"
CORS_ORIGINS="https://bhojrajmishra.com.np,https://www.bhojrajmishra.com.np"
PUBLIC_BASE_URL="https://bhojrajmishra.com.np"
```

```
scp -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes prod.env \
    bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net:/home/bhojrajmishra/portfolio/backend/.env
ssh -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net \
    "chmod 600 /home/bhojrajmishra/portfolio/backend/.env"
```

5.7 — Install backend dependencies on the server

```
ssh -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net \
    "cd /home/bhojrajmishra/portfolio/backend && npm ci --omit=dev"
```

`--omit=dev` skips developer-only tools (TypeScript compiler, etc). The server only needs what's required to *run* the already-compiled code — this also matters because this hosting account has a small file-count quota, and dev tools add thousands of extra files.

5.8 — Apply the database schema and seed starter data

```
ssh -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net \
"cd /home/bhojrajmishra/portfolio/backend && \
mysql -h mysql-bhojrajmishra.alwaysdata.net -u <dbuser> -p'<dbpassword>' <dbname> < sql/schema.sql && \
node dist/src/scripts/seed.js"
```

5.9 — Build and upload the frontend

```
cd frontend
flutter build web --release

# Shrink it: Flutter ships several alternate rendering engines "just in
# case" — only one is actually needed, so delete the rest (~25MB saved,
# meaningful on a small disk quota)
cd build/web/canvaskit
rm -f skwasm*.wasm skwasm*.js skwasm*.js.symbols wimp.wasm wimp.js wimp.js.symbols \
    canvaskit.js.symbols chromium/canvaskit.js.symbols
rm -rf experimental_webparagraph
cd ../../..

tar -C frontend/build/web -czf frontend.tar.gz .
scp -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes frontend.tar.gz \
    bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net:/home/bhojrajmishra/portfolio/frontend.tar.gz
ssh -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net \
"cd /home/bhojrajmishra/portfolio/frontend && tar xzf ../frontend.tar.gz && rm ../frontend.tar.gz"
```

5.10 — Tell alwaysdata what to do with the uploaded files (admin panel — cannot be done over SSH)

1. **Web** → **Sites** → **Add a site** — Type `Node.js`, Command `node`
`/home/bhojrajmishra/portfolio/backend/dist/src/server.js`, Working directory `portfolio/backend`,
Address `bhojrajmishra.com.np/api`.
2. **Web** → **Sites** → **Add a site** — Type `Static files`, directory
`/home/bhojrajmishra/portfolio/frontend`, Address `bhojrajmishra.com.np` (bare, no path — this is the
main site).
3. **Web** → **Sites** → **Add a site** — Type `Static files`, directory
`/home/bhojrajmishra/portfolio/backend/uploads`, Address `bhojrajmishra.com.np/uploads`.
4. **Domains** → attach `bhojrajmishra.com.np` if it isn't already alwaysdata-managed.
5. Let's Encrypt — one toggle, free auto-renewing SSL.

5.11 — Verify

- `https://bhojrajmishra.com.np` loads the public site with a valid padlock.
- `https://bhojrajmishra.com.np/#/admin/login` — sign in.
- Edit content in the admin panel, confirm it appears live.

- Submit the contact form, confirm it shows up in Admin → Messages.

Part 6 — Deployment: Updating the Live Site

Once the Sites above exist, shipping any future change — code or otherwise — is one command from the project root:

```
./deploy/alwaysdata_deploy.sh
```

What it does, every time:

1. Builds the backend (TypeScript → JavaScript)
2. Packages and uploads `dist/` and `sql/`
3. Uploads `package.json` / `package-lock.json` and reinstalls production dependencies on the server (harmless to run even when nothing changed)
4. Builds the frontend (`flutter build web --release`)
5. Strips the unused CanvasKit files (keeps well under the disk quota)
6. Packages and uploads the frontend build

One thing the script cannot do: if you changed backend *code*, the already-running Node process doesn't notice until it's restarted — alwaysdata does not auto-restart on file changes (confirmed directly during this project). After any backend change:

admin.alwaysdata.com → Web → Sites → the `/api` Node.js site → Restart.

The frontend needs no restart — static files are live the instant they're uploaded.

If a change adds a brand-new database table or column, apply that once by hand first (the script never touches the database):

```
ssh -i ~/.ssh/alwaysdata_portfolio -o IdentitiesOnly=yes bhojrajmishra@ssh-bhojrajmishra.alwaysdata.net
mysql -h mysql-bhojrajmishra.alwaysdata.net -u <dbuser> -p'<dbpassword>' <dbname> \
< backend/sql/schema.sql
```

6.1 — Local development (try changes before deploying)

```
# 1. Start a local MySQL database in Docker (mirrors production)
docker compose -f deploy/docker-compose.yml up -d

# 2. Run the backend
cd backend
npm install
npm run dev                # http://localhost:3000, auto-restarts on save

# 3. Run the frontend (separate terminal)
cd frontend
flutter pub get
flutter run -d chrome --dart-define=API_BASE_URL=http://localhost:3000/api
```

This gives you a real browser window showing the site talking to a local database — edit code, save, see it update, with zero risk to the live site. Only run `./deploy/alwaysdata_deploy.sh` once you're happy with a change locally.

6.2 — Changing the admin password

1. SSH in and edit `/home/bhojrajmishra/portfolio/backend/.env`, changing `ADMIN_EMAIL` / `ADMIN_PASSWORD`.
2. Run `node dist/src/scripts/seed.js` — this only updates the admin login, it never touches your content.
3. If you also changed the email to a brand-new address, delete the old row from `admin_users` so there's only one valid login (the seed script upserts by email — it won't remove an old, different email automatically).

Part 7 — Everything Built, in Order (Changelog)

1. **Initial build:** the whole app from scratch — Flutter frontend, Node/MySQL backend, admin panel with CRUD screens for every content type — using Prisma as the database layer initially, verified fully working on a local Docker MySQL setup.
2. **First deploy attempt:** given SSH access to an alwaysdata account. Discovered it's shared hosting (not a VPS) with real constraints — small file-count and disk-space quotas, existing unrelated projects already on the account (a CMS app, a Moodle install, a notes app).
3. **Rewrote the database layer** from Prisma to plain `mysql2` + hand-written SQL, because Prisma's code-generation step didn't fit the account's file-count quota. Along the way, found and fixed a timezone bug where dates were shifting by a day.
4. **Hit the disk quota** mid-upload of the Flutter build. Freed space by deleting the account's old unrelated projects (confirmed explicitly, itemized, before doing anything irreversible) and by stripping unused CanvasKit renderer variants from the Flutter build (~25MB).
5. **Got the site live** at bhojrajmishra.com.np, verified end-to-end on the real domain (login, content edits, contact form).
6. **Redesigned the whole public site** as an interactive macOS-style desktop (menu bar, live clock, live weather widget, desktop icons, dock, draggable/resizable windows) — replacing the original scrolling one-page layout. Deployed and verified live.
7. **Polished the icons/wallpaper** to match the reference design more closely: real brand icons (GitHub/LinkedIn/etc via `font_awesome_flutter`), a live calendar-style date tile, a hand-painted flowing gradient wallpaper, admin-editable badge numbers on social icons (never fabricated — only what you type in yourself). Deployed and verified live.
8. **Changed the admin login** to your real email/password.
9. **Built the Blog feature** from scratch — a new database table, API routes, an admin editor with a Markdown Edit/Preview toggle, and a public "Blog" window styled like a mini browser (address bar, back button) with a modern card list and full Markdown-rendered post view.
10. **Populated the entire site with your real CV content** — Hero, About, all skills grouped by category, both work experiences, education, all 6 real projects, social links, and your actual resume PDF — both locally and on the live site.
11. **Found and fixed a real production bug:** uploaded files (resume, images) were silently being saved to the wrong folder in production (a path-resolution difference between local development and the compiled build), which meant they got wiped on every redeploy. Fixed the root cause in `upload.ts`.
12. **Built the Android APK download feature** — a new database table, API route, admin upload page, and a "Download App" icon in the dock/desktop that only appears once you've actually uploaded an APK.
13. **Wrote a real first blog post** about the site itself and published it live.
14. **Wrote this document** and the deployment reference files in `deploy/`.

Known open item at the time of writing: the `/uploads` static Site (step 5.10 #3 above) needs to be checked/fixed in the admin panel — file uploads (resume, project images, APK) are saved correctly on the server but aren't downloadable yet until that Site is confirmed correctly configured.

Part 8 — Credentials & Reference

What	Where it's stored
Admin panel login	Database (<code>admin_users</code> table) — set via <code>backend/.env</code> + <code>seed.js</code> , never in git
Database password	Only in <code>backend/.env</code> on the server (and your local <code>backend/.env</code> for local dev) — never in git
SSH private key	<code>~/.ssh/alwaysdata_portfolio</code> on this machine only
JWT signing secret	<code>backend/.env</code> only — if it ever leaks, every issued login token should be considered compromised; change it and everyone gets logged out

File map (quick reference)

Path	What it is
README.md	Local dev quickstart
deploy/ALWAYSdata.md	Full deployment reference for this specific hosting account
deploy/HOW_I_DEPLOYED.md	The original beginner walkthrough (this document supersedes it with more detail)
deploy/alwaysdata_deploy.sh	The one-command redeploy script
deploy/docker-compose.yml	Local MySQL for development
deploy/DEPLOY.md , nginx.conf , ecosystem.config.js , deploy.sh	Reference for a <i>different</i> hosting scenario — a traditional VPS with root access. Not used for this deployment, kept in case you ever move to one.
backend/sql/schema.sql	Every database table
backend/.env.example	Template showing every environment variable the backend needs